Software Architecture

SE6103

Sachin Tharaka
(BSc Honours in Software Engineering)

SE6103 - Course Outline

- Course credits- 2 credits
- No. of teaching hours- 30 hours Theory (70 Independent Learning hours)
- Teaching & learning method- Lectures, Discussions
- Assessment- end of course examination, continuous assessments

SE6103 - Recommended Reading

- Frank J. van der Linden, Klaus Schmid and Eelco Rommes, Software product lines in action: The best industrial practice in product line engineering, 2007th ed., Springer, ISBN-13: 978 3540714361, 2007.

- Len Bass, Paul Clements and Rick Kazman, Software Architecture in Practice, 3rd ed., Pearson, ISBN-13: 9780321815736, 2012.

SE6103 – Course Content

- Topic 01: Basic concepts & principles about software architecture
- Topic 02: Introduction to Software Architectural pattern
- Topic 03: ADL
- Topic 04: 4+1 Architecture
- Topic 05: Practical approaches & methods for Create & Analyse software architecture
- Topic 06: Quality attributes of software architectures
- Topic 07: Examples in architectural design applications & case studies in software architecture (N tier architecture, SOA, Cloud, etc.)

Software Architecture

Software Architecture

Software Architecture focuses on:

- High-level structure of software systems
- Design decisions that shape the system
- Quality attributes (performance, security, scalability)
- Architectural patterns and styles
- Documentation using UML
- Evaluating and analysing architectures

Why this course matters

"Most software failures are NOT coding failures. They are architectural failures."

We should,

- Think beyond code
- Design large, scalable systems
- Make strategic technical decisions

What is Software Architecture?

According to Software Architecture in Practice by Len Bass, Paul Clements, and Rick Kazman

"Software architecture is the structure or structures of the system, which comprise software elements, externally visible properties of those elements, and the relationships among them."

Architecture

- Architecture means Structure
- Includes components and connectors
- Includes externally visible behavior
- Includes constraints and rationale

Architecture vs Design vs Code

Level	Focus	Example
Code	Implementation	Java class
Design	Component logic	Payment module
Architecture	System structure	3-tier web system

Architecture - Why its important

Architecture decisions are

- Hard to change later
- Expensive to refactor
- Impact system quality

Architecture determines

- System scalability
- Performance
- Security
- Maintainability
- Deployability
- Testability

Architecture - Why its important

If architecture is poor

- The system becomes rigid
- Maintenance cost increases
- Performance bottlenecks appear
- Scaling becomes expensive

Real World Example: Architecture Failure

Case: Early monolithic e-commerce platforms

Problem

- All modules tightly coupled
- One bug affects entire system
- Cannot scale independently

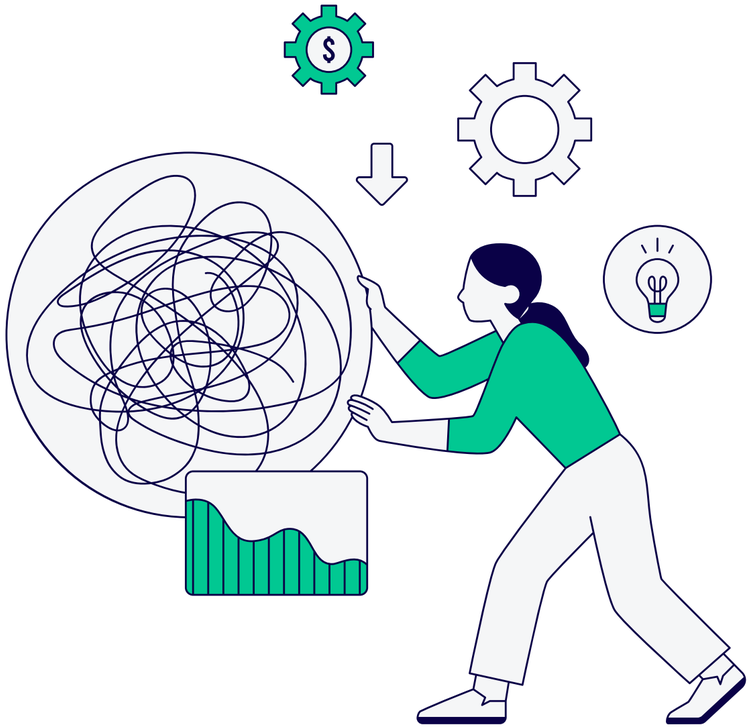
Modern systems like Amazon moved to

- Service-based architecture
- Independent deployment
- Horizontal scaling

Stakeholders in Software Architecture

Architecture serves multiple stakeholders

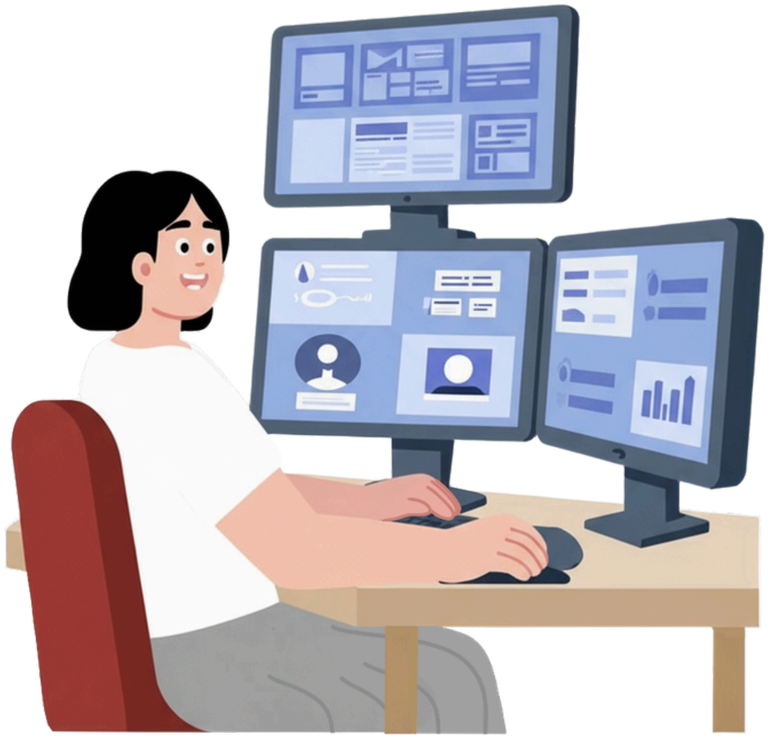
- Developers
- Project managers
- System administrators
- Business owners
- Clients
- End users



Different PoVs as **Developer** cares about modularity. **Businesses** care about cost and scalability.

Multiple Perspectives in Architecture

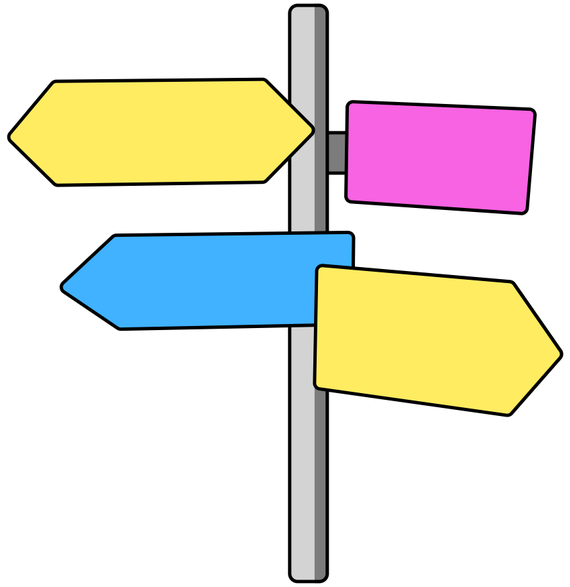
- Logical view (functional elements)
- Development view (module organization)
- Process view (runtime behavior)
- Physical view (deployment)



Architectural Decisions

Architecture includes

- Technology stack
- Communication style (REST, messaging)
- Data storage model
- Deployment model
- Security mechanisms



Architectural decisions are

- Strategic
- Long-term
- Costly to reverse

Functional vs Non-Functional Requirements

Functional Requirements

What system does,

- Process payment
- Register user

Non-Functional Requirements

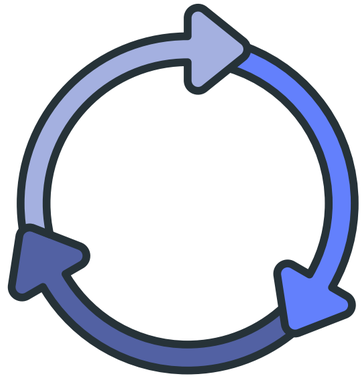
How the system behaves,

- Performance
- Security
- Availability
- Scalability
- Reliability

Quality Attributes Overview

- Performance
- Scalability
- Security
- Modifiability
- Availability
- Testability

Architecture trade-offs are inevitable.



Architecture Trade-offs

There is no perfect architecture.

Architectural Styles vs Patterns

Architectural Style: General category

- Ex: Layered architecture

Architectural Pattern: Reusable solution to a common problem

- MVC
- Microservices
- Client-Server

Basic Architectural Styles

- Layered architecture
- Client-server
- Event-driven
- Microservices
- Service-oriented architecture (SOA)

Architecture Description Languages (ADL)

Architecture must be documented.

ADL helps

- Describe components
- Describe connectors
- Define constraints
- Enable analysis

Architecture Documentation

Architecture is documented using,

- UML diagrams
- Component diagrams
- Deployment diagrams
- Sequence diagrams
- Views & viewpoints

Architecture and Risk

- Technical risks
- Scalability risks
- Security vulnerabilities
- Performance bottlenecks

Architecture and Business Strategy

Architecture supports

- Time to market
- Product evolution
- Cost control
- Technology adaptation

Architecture in Modern Systems

Modern systems include

- Cloud-based systems
- Distributed systems
- Mobile systems
- IoT systems

Cloud providers like Amazon Web Services promote scalable architecture models.

Architecture Evaluation

Common evaluation approaches:

- Scenario-based evaluation
- Trade-off analysis
- Quality attribute analysis

You will learn how to:

- Compare architectures
- Justify decisions
- Critically analyse systems

Architecture vs Implementation Thinking

“If the code works, the system is good.”

- Reality: System may Fail under load, Be insecure, Be impossible to maintain
- Architecture thinking is long-term thinking

Course Roadmap

- Fundamentals
- Architectural patterns
- ADL
- 4+1 model
- Practical design approaches
- Quality attributes
- Case studies (N-tier, SOA, Cloud)

Activity

- Why did Facebook move from monolithic PHP to a distributed architecture?
- What happens if a banking system has poor availability?
- Which is more important: performance or security?

Key Takeaways

- Software architecture defines system structure
- Influences quality attributes
- Impacts business success
- Is difficult to change
- Must be documented and evaluated

Thank You!

Evolution of Software Architectures